

DOCUMENTO TÉCNICO DE ARQUITECTURA Y DISEÑO

TRABAJO FINAL - PROGRAMACIÓN DISTRIBUIDA Y CONCURRENTE (PDyC 2026)

Institución: Universidad Nacional del Noroeste de la Prov. de Bs. As. (UNNOBA)

Carrera: Ing. en Informática

Modalidad: Individual (Alumno: Bruno Bertacchini)

Unidad Académica: Escuela de Tecnología

Asignatura: Programación Distribuida y Concurrente (PDyC)

Mejora Seleccionada: Opción F – Calidad (Testing Automatizado)

1. INTRODUCCIÓN Y ALCANCE DEL SISTEMA

1.1 Contexto y Problemática Distribuida

Los sistemas modernos de catálogo de espectáculos masivos y redes sociales asociadas enfrentan desafíos críticos de concurrencia, disponibilidad, escalabilidad y consistencia de datos. En las arquitecturas monolíticas tradicionales, los picos de demanda generados por anuncios de venta de entradas o cancelaciones masivas degradan el rendimiento general del sistema, produciendo bloqueos en bases de datos compartidas y acoplando dominios disímiles como la gestión administrativa del catálogo, las interacciones sociales de los usuarios y el despacho de notificaciones.

Greater Events Microservices es una plataforma distribuida concebida para resolver esta problemática, desacoplando los dominios funcionales en microservicios autónomos que se comunican mediante protocolos especializados: HTTP/REST sincrónico, mensajería asíncrona AMQP sobre RabbitMQ y llamadas remotas binarias de alta velocidad con gRPC sobre HTTP/2.

1.2 Requerimientos del Sistema

Requerimientos Funcionales:

- Catálogo de Artistas y Eventos:** Alta, modificación, control de ciclo de vida del evento (**TENTATIVE** , **CONFIRMED** , **RESCHEDULED** , **CANCELLED**), asignación y desvinculación de artistas.
- Interacción Social:** Capacidad de los usuarios autenticados de seguir/dejar de seguir artistas, guardar eventos como favoritos y consultar la cartelera de sus artistas seguidos.
- Identidad y Control de Acceso:** Autenticación centralizada con protocolo OpenID Connect / OAuth2 sobre Keycloak y control de acceso basado en roles (**ROLE_admin** para la gestión administrativa).
- Notificaciones Automatizadas:** Despacho automatizado, desacoplado y confiable de notificaciones a todos los usuarios interesados ante la cancelación de un evento musical.

Atributos de Calidad Distribuida (No Funcionales):

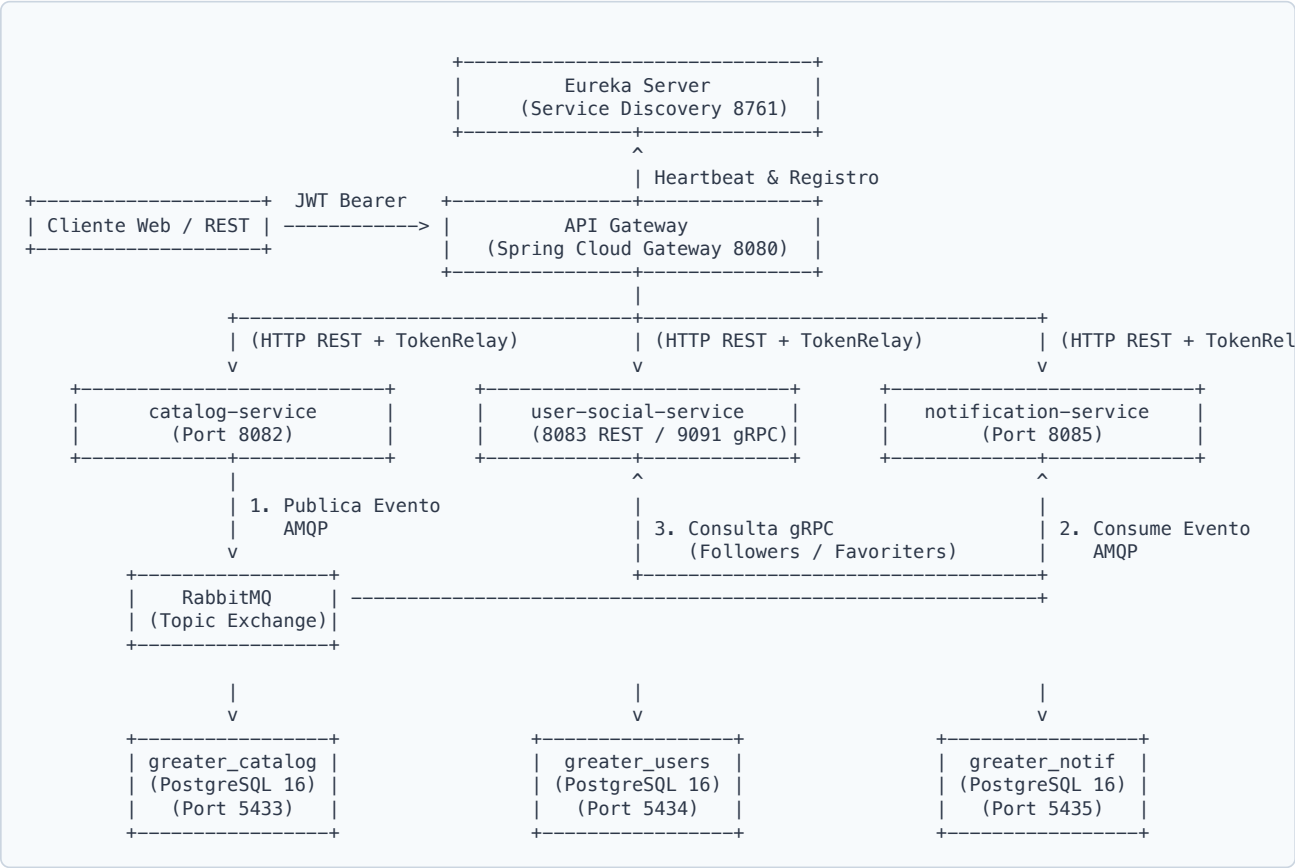
- Desacoplamiento Temporal:** La cancelación de un evento no bloquea el hilo de ejecución esperando notificar a miles de usuarios.
- Aislamiento de Fallos y Datos (Database per Service):** La indisponibilidad de una base de datos o servicio no compromete la operatividad de los demás módulos.
- Baja Latencia Inter-Servicio:** Consultas internas de alto volumen se resuelven con protocolos binarios de alta eficiencia (gRPC sobre HTTP/2).

- **Portabilidad y Despliegue:** Infraestructura y servicios reproducibles en cualquier entorno mediante contenedores Docker y Docker Compose.
- **Calidad y Verificabilidad Rigurosa:** Cobertura de 42 pruebas automatizadas que garantizan la solidez de las reglas de negocio y contratos de integración.

2. ARQUITECTURA GENERAL Y TOPOLOGÍA DEL SISTEMA

El sistema implementa el estilo arquitectónico de microservicios sobre la pila tecnológica **Spring Cloud 2024.0.1** y **Spring Boot 3.4.5**, ejecutada sobre **Java 17**.

2.1 Diagrama de Arquitectura y Comunicación Global



2.2 Inventario de Componentes y Puertos

Módulo / Contenedor	Puerto Host	Puerto Cont.	Protocolo	Responsabilidad Técnica
api-gateway	8080	8080	HTTP/1.1 (WebFlux)	Punto de entrada único, enrutamiento dinámico, retransmisión de tokens JWT (<code>TokenRelay</code>).
eureka-server	8761	8761	HTTP/1.1	Registro y catálogo dinámico de instancias vivas (<i>Service Discovery</i>).
config-server	8888	8888	HTTP/1.1	Servidor de configuración centralizada nativa (<code>classpath:/config</code>).
catalog-service	8082	8082	HTTP/1.1 (MVC)	Catálogo de artistas y eventos. Publicador de eventos AMQP.
user-social-service	8083	8083	HTTP/1.1 (MVC)	Seguimiento de artistas, eventos favoritos y administración de Keycloak.
user-social-service (gRPC)	9091	9091	gRPC sobre HTTP/2	Servidor RPC binario de alta velocidad para consultas internas de relaciones sociales.
notification-service	8085	8085	HTTP/1.1 (MVC)	Consumidor AMQP, cliente gRPC y consulta de notificaciones de usuario.
Keycloak IAM	8084	8080	HTTP/1.1	Identity Provider OAuth2 / OpenID Connect (Realm <code>unnoba</code>).
RabbitMQ Broker	5672 / 15672	5672 / 15672	AMQP 0-9-1 / HTTP	Broker de mensajería orientada a eventos y consola de administración web.
PostgreSQL Catalog	5433	5432	PostgreSQL Wire	Base de datos relacional de catálogo (<code>greater_catalog</code>).
PostgreSQL Users	5434	5432	PostgreSQL Wire	Base de datos relacional social y referencias de usuarios (<code>greater_users</code>).
PostgreSQL Notifications	5435	5432	PostgreSQL Wire	Base de datos relacional de notificaciones (<code>greater_notifications</code>).

3. DECISIONES DE DISEÑO Y PATRONES DISTRIBUIDOS

3.1 Patrón *Database per Service*

Compartir una base de datos única introduce un único punto de fallo (SPOF), genera cuellos de botella de concurrencia y acopla los esquemas de persistencia. En **Greater Events**, cada microservicio es dueño exclusivo de su esquema físico en PostgreSQL.

Manejo de relaciones cruzadas sin JOINS distribuidos: `user-social-service` no guarda llaves foráneas hacia las tablas de catálogo; persiste identificadores numéricos (`Set<Long> followedArtists` , `Set<Long> favoriteEvents`). Para renderizar vistas enriquecidas con nombres y fechas, utiliza **OpenFeign** para consultar sincrónicamente al `catalog-service` y componer los DTOs en memoria.

3.2 Patrón *API Gateway* y Seguridad con *Token Relay*

Los clientes externos solo acceden a través del Gateway (puerto 8080). El Gateway aplica el filtro `TokenRelay` , retransmitiendo el encabezado `Authorization: Bearer <JWT>` hacia los microservicios subyacentes. Cada microservicio actúa como un **OAuth2 Resource Server** que valida la firma criptográfica RSA contra el endpoint JWKS de Keycloak (`jwk-set-uri`) y extrae los roles mediante `KeycloakJwtGrantedAuthoritiesConverter` (ej. `ROLE_admin`).

3.3 Arquitectura Orientada a Eventos con RabbitMQ

Al cancelar un evento en `catalog-service` , se emite un mensaje a la Topic Exchange `events.exchange` con routing key `event.cancelled` . La cola `event.cancelled.queue` es durable (`durable = true`). Si `notification-service` está temporalmente caído, los mensajes se retienen de forma segura en RabbitMQ y se consumen automáticamente cuando el servicio se recupera, garantizando **consistencia eventual**.

3.4 RPC Binario de Alto Rendimiento (gRPC sobre HTTP/2)

Para recuperar en milisegundos los miles de usuarios que siguen a un artista o marcaron un evento favorito, `notification-service` consulta al `user-social-service` vía **gRPC** (puerto 9091). Gracias a HTTP/2 multiplexado y la serialización binaria con **Protocol Buffers**, el tamaño del payload y la latencia de red se reducen drásticamente en comparación con REST/JSON.

4. MODELO DE DATOS Y REGLAS DE NEGOCIO

- **Ciclo de Vida de Eventos:** Se inician en `TENTATIVE` (único estado que admite modificaciones o borrado). Pasan a `CONFIRMED` únicamente si su fecha es estrictamente futura. Pueden reprogramarse a `RESCHEDULED` (exigiendo nueva fecha futura) o cancelarse a `CANCELLED` (disparando el evento AMQP).
- **Inmutabilidad Histórica de Artistas:** Un artista con eventos asociados no puede editarse ni eliminarse físicamente para no corromper registros históricos; se aplica borrado lógico (`active = false`).
- **Regla de Favoritos:** Solo pueden guardarse como favoritos eventos en estado `CONFIRMED` o `RESCHEDULED` con fecha futura.

5. EVOLUCIÓN DEL SISTEMA: OPCIÓN F – CALIDAD Y PRUEBAS AUTOMATIZADAS

Como mejora de evolución individual, se diseñó e implementó una **estrategia de calidad integral basada en la Pirámide de Pruebas**, alcanzando **42 pruebas automatizadas** que garantizan el correcto funcionamiento del sistema sin depender de infraestructura pesada.

5.1 Justificación Teórica: Tests Unitarios Puros vs. Tests de Integración

En microservicios, depender únicamente de pruebas manuales o de tests con `@SpringBootTest` introduce dos desventajas críticas:

1. **Lentitud:** Levantar Spring Boot, escanear componentes e inicializar conexiones tarda varios segundos por clase.
2. **Imposibilidad de simular fallos:** Probar cómo responde el sistema si un servicio externo cae (ej. Feign con timeout o error 500) es difícil con servicios reales.

Por eso implementamos una arquitectura en dos capas:

- **34 Tests Unitarios Puros (JUnit 5 + Mockito):** Corren en memoria pura de la JVM en menos de 200 milisegundos. Aíslan la clase bajo prueba y reemplazan repositorios, clientes Feign, stubs gRPC y RabbitTemplate por Mocks programables.
- **8 Tests de Integración (Spring Boot + H2 en memoria):** Ejecutan transacciones SQL reales, verifican mapeos Hibernate y validan persistencia sin requerir bases de datos PostgreSQL externas.

5.2 Desglose Exhaustivo de las 42 Pruebas Automatizadas

Microservicio	Clase de Prueba	Tipo	Tests	Aspectos y Reglas de Negocio Verificadas
catalog-service (25 tests)	ArtistServiceTest	Unitario (Mockito)	9	Creación válida, validación de nombre/género nulo (<code>BusinessException</code>), búsqueda 404, filtrado, inmutabilidad de artistas con eventos, borrado físico vs. lógico (<code>active = false</code>).
	EventServiceTest	Unitario (Mockito)	11	Creación tentativa, bloqueo de edición en no-tentativo, rechazo de artistas inactivos, confirmación con fecha futura vs. pasada, reprogramación, cancelación y verificación AMQP con <code>verify(rabbitTemplate)</code> .
	CatalogServiceIntegrationTest	Integración (H2)	5	Persistencia relacional JPA en H2, transacciones reales, rollback, unión Many-to-Many entre eventos y artistas.
user-social-service (10 tests)	UserServiceTest	Unitario (Mockito)	8	Seguir artista activo con mock de Feign, captura de fallos de catálogo (404), bloqueo a artistas inactivos, unfollow no seguido, validación estricta de favoritos (solo futuros confirmados).
	UserSocialServiceIntegrationTest	Integración (H2)	2	Persistencia de colecciones elementales JPA (<code>@ElementCollection</code> en <code>user_followed_artists</code> y <code>user_favorite_events</code>).
notification-service (7 tests)	NotificationServiceTest	Unitario (Mockito)	4	Listado de leídas/no leídas, marcado como leída, seguridad y aislamiento (lanza <code>AccessDeniedException</code> si se altera notificación ajena).
	EventCancelledListenerTest	Unitario (Mockito)	2	Desduplicación estricta mediante <code>ArgumentCaptor</code> para evitar notificaciones duplicadas ante seguidores/favoritos cruzados; resiliencia ante listas nulas de artistas.
	NotificationServiceIntegrationTest	Integración (H2)	1	Flujo end-to-end de consumo AMQP, resolución gRPC simulada y persistencia de notificaciones en H2.

5.3 Resolución de Compatibilidad con Java Moderno

Al ejecutar en entornos con Java 25, ByteBuddy requiere autorización explícita para instrumentar código dinámicamente en tiempo de ejecución. Se parametrizó Maven pasando `-Dnet.bytebuddy.experimental=true` , garantizando una ejecución limpia de Mockito sin advertencias del ClassLoader.

6. INFRAESTRUCTURA Y GUÍA DE DESPLIEGUE

Para satisfacer el requisito de *"Despliegue funcional de todos los servicios"*, se configuró la orquestación completa en **Docker Compose** (11 contenedores) con Dockerfiles basados en `eclipse-temurin:17-jre` (<200MB por imagen) y sincronización con healthchecks activos (`pg_isready` y `rabbitmq-diagnostics ping`).

```
# 1. Ejecutar las 42 pruebas automatizadas
mvn clean test -DargLine="-Dnet.bytebuddy.experimental=true"

# 2. Compilar paquetes JAR
mvn clean package -DskipTests

# 3. Desplegar todo el ecosistema con Docker Compose
docker compose up --build -d

# 4. Verificar contenedores
docker compose ps
```

7. CONCLUSIÓN

La plataforma **Greater Events Microservices** satisface rigurosamente los principios de diseño de sistemas distribuidos modernos. La incorporación de la **Opción F (Estrategia Integral de Pruebas Automatizadas)** aporta un estándar profesional de ingeniería de software, respaldado por una suite reproducible de 42 pruebas que garantizan la robustez de la solución presentada.